

OLEG SAFRONOV

Full-cycle engineer
Senior Backend · AI Integration

→ Production-ready systems from scratch:
architecture → code → deployment → security

SPECIALTY
AI/LLM integrations · Multi-model pipelines · Business process automation

4 YEARS COMMERCIAL	4 CONTRACTS 5.0 RATING	4 CLIENT COUNTRIES	5 PORTFOLIO PROJECTS
--	--	--	--

USA · United Kingdom · Canada · Poland

TELEGRAM
@safronov_oleg

EMAIL
safronovoleg1@gmail.com

GITHUB
safronovlab



Senior Backend engineer working in Python. I cover the full cycle: design, code, tests, deployment, monitoring, support. 4 years of commercial experience, 5 completed projects for clients in the USA, United Kingdom, Canada and Poland.

How I work. Documentation and tests come first, then code. Every project ships with a runbook.

Backend and architecture. Python (FastAPI, Django, Flask), async I/O on asyncpg, httpx, aiohttp, aiogram, uvloop. Queues: Celery, RQ, Dramatiq, Redis, RabbitMQ. APIs: REST, gRPC, GraphQL, WebSockets. Auth: JWT, OAuth2, OpenID Connect. Clean Architecture, Hexagonal, DDD where it fits.

Data and quality. PostgreSQL, MySQL, ClickHouse, MongoDB, Redis, S3. Vector DBs for RAG: pgvector, Pinecone, Weaviate, Chroma. Tests on pytest up to 320 per project, static analysis (mypy strict, pyright, ruff, bandit), pre-commit hooks, PR review.

AI and agents. I work with Anthropic Claude (Opus, Sonnet, Haiku) through Claude API, Claude Code and Claude Agent SDK. I set up MCP servers. I build multi-agent systems with tool use and function calling in production. RAG with vector DBs, two-pass filtering, prompt caching, quality and cost evaluation against metrics. I integrate AI into existing products and business processes.

Frontend and specialties. React 19, Next.js 14, TypeScript, Tailwind, PWA when the project needs full-stack work. Computer Vision: OpenCV, scikit-image, LAB color, Delta-E. Web3: Polkadot.js, Bittensor RPC.

DevOps. I handle infrastructure myself. AWS, Hetzner, DigitalOcean, Docker, basic Kubernetes, Terraform, Ansible. Reverse proxy: nginx, Traefik. CI/CD: GitHub Actions, GitLab CI.

Security. Background in information security (Russian University of Transport, honors degree). I apply it on security-critical work: Fernet/AES encryption, HMAC webhook signatures, RLS policy audits, threat modeling, server hardening (SSH keys, fail2ban, ufw, AppArmor), secret management with Vault and SOPS.

Rust. Performance-critical modules in Rust (Tokio, Serde).



03

Technology stack

Backend

PRIMARY EXPERTISE

Python 3.12

FastAPI

Django

Flask

aiogram 3

aiohttp

httpx

asyncpg

SQLAlchemy

Tortoise ORM

Pydantic v2

msgspec

uvloop

Clean Architecture

APIs · Queues · Brokers

INTEGRATION AND ASYNC

REST

gRPC

GraphQL

WebSockets

Server-Sent Events

Celery

RQ

Dramatiq

RabbitMQ

Redis Pub/Sub

JWT

OAuth2

OpenID Connect

Frontend

FULL-STACK WHEN NEEDED

React 19

Next.js 14

TypeScript

Tailwind CSS

Vite

TanStack Query

Web Workers

PWA

Recharts

AI · Agents

SPECIALTY

Anthropic Claude

Claude Code

Claude Agent SDK

MCP servers

Tool use

Function calling

Multi-agent

OpenRouter

RAG

pgvector

Prompt eval

Business automation

Data · Storage

DATABASES AND CACHING

PostgreSQL

MySQL

ClickHouse

MongoDB

Redis

Upstash

SQLite (WAL)

Supabase

S3 / MinIO

Alembic

DevOps · Infrastructure

I HANDLE INFRA MYSELF

AWS

Docker

Kubernetes

Terraform

Ansible

Hetzner

DigitalOcean

Coolify

Vercel

nginx

Traefik

Caddy

Linux / Ubuntu

Let's Encrypt

CI/CD · Observability

PRODUCTION MATURITY

GitHub Actions

GitLab CI

Prometheus

Grafana

Loki

Sentry

OpenTelemetry

Structured logging

pre-commit hooks

Security · Quality

BACKGROUND: CYBERSECURITY

Fernet / AES

Server hardening

mypy strict

pyright

ruff

bandit

RLS audit

Threat modeling

HMAC / Webhooks

Vault / SOPS

AI Gateway

Async API gateway between business logic and LLM providers

TECH COMPANY • B2B SAAS

THE BRIEF

The client needed a single intelligent proxy layer between their internal services and external LLM providers (through Portkey). Goals: centralized logging, guardrail policies, key encryption, web admin UI. No existing codebase. Started from an empty folder.

WHAT I BUILT

- **Designed Clean Architecture** with 4 layers (domain → services → infrastructure → api). Each layer free of external dependencies.
- **Implemented async API end-to-end** on FastAPI + httpx + SQLAlchemy Async + aiosqlite (PostgreSQL-ready).
- **Added Fernet encryption** for provider API keys in the database. Keys never sit in plaintext.
- **Built the guardrail system** with policies, cloud sync, runtime enable / disable.
- **Built the web admin UI** for managing providers, policies, log review and statistics.
- **Deployed on AWS** (EC2, security groups, SSL, monitoring) plus webhook signature verification through X-Webhook-Secret. Handed over full access and runbook.
- **Covered the code with ~120 tests** (unit + integration), pre-commit hooks with mypy strict, ruff, bandit. Tests written before code based on the contract in ARCHITECTURE.md.

OUTCOME

~120 TESTS UNIT + INTEGRATION	4 ARCHITECTURE LAYERS CLEAN ARCHITECTURE	100% PRODUCTION-READY AWS + UI + DOCS
---	--	---

"Quality work delivered, on time. Oleg is great to work with."

CLIENT REVIEW • 5.0 / 5.0 • AWS / DEVOPS ENGAGEMENT

Bulk Recolor

Computer vision tool for batch design recoloring

STREETWEAR / SNEAKER BRAND • E-COMMERCE

THE BRIEF

The client (a streetwear brand with weekly drops) had a v1 recolor tool that broke on three categories of images. The job: replace it with v2 that fixes all three failures and runs in production. Fixed-price contract.

WHAT I BUILT

- **Diagnosed three failure modes** in v1: whole-image bleed, dotted edge artifacts, cross-design hex drift.
- **Built three algorithms** with regression tests on the 6 previously failing designs:
 - Connected-region flood fill (cv2.floodFill) instead of global hex match: bleed fixed.
 - Edge-aware blending that respects alpha boundaries: fringe artifacts fixed.
 - LAB-space K-means clustering + Delta-E 2000 palette mapping: cross-design drift fixed.
- **Built the full-stack app:** FastAPI backend + React 19 frontend (Vite, Tailwind) + RQ queue on Redis + nginx reverse proxy.
- **Added click-fix mode:** user clicks the bad region, backend flood-fills just that connected component.
- **Covered the code with ~320 tests** (unit + integration + acceptance) and shipped everything as a Docker Compose stack.

OUTCOME

~320 TESTS UNIT + INTEGRATION	<60s END-TO-END 20 DESIGNS ON 2 GB VPS	<200ms LATENCY CLICK-FIX OPERATION
---	---	---

"Oleg picked up where our previous developer left off and delivered a much more reliable tool. He took the time to understand the specific failure modes, asked the right questions, and built something that actually works for our workflow."

CLIENT REVIEW • 5.0 / 5.0

TaoScope

Real-time Bittensor blockchain explorer

WEB3 / BLOCKCHAIN • LIVE: TAOSCOPE.COM

THE BRIEF

The client needed a full-featured public explorer for the Bittensor network with real-time data on subnets, validators, miners, prices and historical metrics. From scratch, first line of code to an optimized production deployment accessible globally.

WHAT I BUILT

- **Built a Next.js 14 app** with App Router on TypeScript: 10+ pages, server API routes, PWA with offline mode.
- **Integrated the blockchain:** WebSocket client to Bittensor mainnet through Polkadot.js API, custom Bittensor types, auto-reconnect with fallback RPC.
- **Set up dual-layer caching:** Upstash Redis (edge) + in-memory fallback. TTLs per data type: 30s / 60s / 120s / 300s.
- **Connected 3 wallets:** Polkadot.js, Talisman, SubWallet. USD portfolio, stake balance, delegations.
- **Added historical charts** for price and emissions (24h / 7d / 30d / 90d / 1y), delegation calculator with compound interest.
- **Shipped to production through Vercel** (PWA-installable, service worker, offline fallback) and covered the code with ~80 tests (Vitest unit + Playwright e2e), TypeScript strict, ESLint. API route contracts documented before implementation.

OUTCOME

100+ SUBNETS REAL-TIME MONITORING	10+ PAGES INCLUDING WALLET	LIVE TAOSCOPE.COM RUNNING IN PRODUCTION
---	--	---

"Oleg shipped the full Bittensor explorer on time. The Polkadot.js integration was the tricky part and he handled it cleanly. Site has been running in production for months with minimal maintenance on our side."

CLIENT REVIEW • 5.0 / 5.0

Production Incident Response • SEV-1

Forensic diagnosis and recovery plan for a revenue decline

COMMUNITY SAAS PLATFORM • 180K+ USERS

THE BRIEF

The client experienced an unexpected revenue decline of 50 to 60 percent over 35 days. The source was unclear: SEO problem, technical failure, or an algorithmic penalty from Google. I was hired to run the forensic investigation and deliver an engineering recovery plan.

WHAT I DID

- **Ran root cause analysis** across the Next.js + Supabase + Netlify + Klaviyo + Stripe stack.
- **Identified two independent root causes** overlapping in the same time window:
 - Phase 1 (35 days): failed production build, frontend stuck in infinite loading.
 - Phase 2 (6 days): legacy Supabase API keys auto-disabled, Edge Functions stopped working (100% failure rate).
- **Audited Supabase RLS policies**, inventoried Edge Function secrets, reviewed failed Stripe captures.
- **Correlated GSC data**, server logs, edge logs and Stripe logs against the incident timeline.
- **Delivered the Diagnostic Report**: 4 sections + 4 appendices, repair strategy, SRE risk assessment.

OUTCOME

2 INDEPENDENT ROOT CAUSES	41 DAYS ANALYZED PHASE 1 + PHASE 2	180K+ USERS PLATFORM RECOVERY PLAN DELIVERED
---	--	--

“Could not recommend Oleg enough. He understood the issue, was fast, precise, and knew exactly which parts of our infrastructure needed to be investigated to solve our revenue issue.”

CLIENT REVIEW • 5.0 / 5.0

AI Lead Filter Bot

Multi-model AI engine for filtering incoming leads with Telegram delivery

TECH CLIENT • SALES AUTOMATION

THE BRIEF

The client needed an AI system for automatic filtering of incoming leads from an external source, with two-stage analysis to save on tokens, delivering relevant results into Telegram. It had to fit a minimal VPS, including full server setup and security hardening.

WHAT I BUILT

- **Designed a two-pass AI pipeline** with different models per stage:
 - Stage 1: cheap / fast model, rough filtering of clearly irrelevant leads.
 - Stage 2: premium model, deep analysis only on items that passed stage 1 (~20-30% of traffic).
- **Performance engineering to the limit:** asyncpg without ORM (binary protocol, prepared statements), msgspec instead of Pydantic (×3 faster), uvloop, slotted-frozen dataclasses.
- **Single HTTP stack:** aiogram serves the built-in aiohttp.web for both Telegram polling and webhook (no FastAPI on top).
- **Covered the code with ~140 tests** (unit + integration through testcontainers with real Postgres). Tests written before code based on PIPELINE.md and DATABASE.md. Static analysis: mypy strict, ruff, bandit, radon, vulture, xenon.
- **Deployed on VPS** with Docker multi-stage + Coolify, full server hardening, runbook for operations.

OUTCOME

50 MB RESIDENT MEMORY IN PRODUCTION	<50ms P99 WEBHOOK INGESTION	<30s P95 END-TO-END THROUGH LLM STAGES
---	--	---

“Solid execution end-to-end. Oleg owned the architecture, the code, and the deployment. The bot runs on minimal resources and the runbook he left made handoff smooth. Easy to work with across time zones.”

CLIENT REVIEW • 5.0 / 5.0

09 Client testimonials

4 contracts · 5.0 average · all positive reviews

★★★★★ 5.0

FORENSIC SEARCH ANALYST FOR REVENUE & GOOGLE TRAFFIC DIAGNOSIS

Could not recommend Oleg enough. He understood the issue, was fast, precise, and knew exactly which parts of our infrastructure needed to be investigated to solve our revenue issue.

Solution Oriented

Accountable for Outcomes

Detail Oriented

CLIENT REVIEW · UK · APR 2026

★★★★★ 5.0

TOOL DEVELOPMENT FOR BULK IMAGE UPLOAD AND COLOR MANAGEMENT

Oleg picked up where our previous developer left off and delivered a much more reliable tool. He took the time to understand the specific failure modes, asked the right questions, and built something that actually works for our workflow.

Reliable

Detail Oriented

Accountable for Outcomes

CLIENT REVIEW · USA · APR 2026

★★★★★ 5.0

AWS · DEVOPS ENGINEERING

On time deliverable with quality.

Reliable

Committed to Quality

Accountable for Outcomes

CLIENT REVIEW · USA · APR 2026

★★★★★ 5.0

AWS · DEVOPS ENGINEERING

Quality work delivered, on time. Oleg is great to work with.

Reliable

Collaborative

Committed to Quality

CLIENT REVIEW · USA · MAR 2026

READY TO DISCUSS YOUR PROJECT

Get in touch

TELEGRAM

@safronov_oleg

EMAIL

safronovoleg1@gmail.com

GITHUB

github.com/safronovlab